

Refatoração de Testes e Detecção de Test Smells

PUC Minas - Engenharia de Software

Disciplina: Testes de Software

Trabalho: Refatoração de Testes e Detecção de Test Smells

Nome: João Paulo Gobira

Matrícula: 859611

1. Análise de Smells

Durante a análise manual da suíte de testes original, foram identificados diversos problemas arquiteturais. Abaixo estão descritos três dos principais *Test Smells* encontrados e os riscos associados a eles:

- **Lógica Condicional (Conditional Logic):** Encontrado no teste que valida a desativação de usuários, onde foi utilizado um laço iterativo `for` combinado com uma estrutura condicional `if/else`.
Risco: Testes devem ser lineares e previsíveis. Ao introduzir lógica de programação no próprio teste, cria-se a possibilidade de fluxos inteiros não serem atingidos por conta de um bug. Se o `if` falhar silenciosamente, as asserções (`expect`) são ignoradas e o teste passa com um falso positivo, gerando uma falsa sensação de segurança.
- **Asserção Silenciosa / Captura Incorreta de Exceção (Silent Assertion):** Presente no teste que tenta criar um usuário menor de idade, o qual foi construído utilizando um bloco `try/catch` tradicional.
Risco: Se o método principal apresentar um defeito e parar de lançar a exceção esperada para usuários menores de idade, o código simplesmente não entrará no bloco `catch`. Nenhuma asserção será executada, e o executor (Jest) marcará o teste como um sucesso absoluto, ocultando um defeito crítico na validação de negócio.
- **Teste Frágil / Especificação Excessiva (Fragile Test / Overspecification):** Observado no teste de geração de relatório, que validava a string exata formatada para a saída, exigindo compatibilidade de quebras de linha (`\n`) e traços visuais.
Risco: O teste torna-se engessado e altamente acoplado a detalhes estéticos. Qualquer alteração inofensiva no layout ou design do relatório irá quebrar o teste, resultando em um grande esforço de manutenção contínua, mesmo que a função continue buscando e exibindo os usuários corretamente.

2. Processo de Refatoração

O teste escolhido para demonstrar o processo de limpeza foi o de **desativação de usuários**, pois ele sofria gravemente com o smell de Lógica Condicional e misturava diferentes intenções de teste dentro do mesmo bloco.

Antes (Código Smelly)	Depois (Código Clean)
<pre>test('deve desativar usuários se eles não forem administradores', () => { const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30); const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true); const todosOsUsuarios = [usuarioComum, usuarioAdmin]; for (const user of todosOsUsuarios) { const resultado = userService.deactivateUser(user.id); if (!user.isAdmin) { expect(resultado).toBe(true); const usuarioAtualizado = userService.getUserById(user.id); expect(usuarioAtualizado.status).toBe('inativo'); } else { expect(resultado).toBe(false); } } });</pre>	<pre>test('deve desativar um usuário comum com sucesso', () => { // Arrange const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30); // Act const resultado = userService.deactivateUser(usuarioComum.id); const usuarioAtualizado = userService.getUserById(usuarioComum.id); // Assert expect(resultado).toBe(true); expect(usuarioAtualizado.status).toBe('inativo'); }); test('não deve permitir a desativação de um usuário administrador', () => { // Arrange const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true); // Act const resultado = userService.deactivateUser(usuarioAdmin.id); const usuarioAtualizado = userService.getUserById(usuarioAdmin.id); // Assert expect(resultado).toBe(false); expect(usuarioAtualizado.status).toBe('ativo'); });</pre>

Decisões Tomadas:

- **Eliminação da Lógica Condicional:** O laço for e as ramificações if/else foram completamente removidos. Em vez de iterar sobre perfis diferentes com comportamentos distintos, o teste complexo foi dividido em dois testes independentes, focados e diretos.

- **Aplicação do Padrão AAA:** Os novos testes adotam explicitamente o padrão estrutural *Arrange, Act, Assert*, utilizando comentários para separar de maneira visual a preparação do estado, a execução da ação principal e a verificação do resultado.
- **Nomenclatura Clara e Descritiva:** Os nomes genéricos foram substituídos por frases completas que descrevem com exatidão o cenário de entrada e o resultado esperado, agindo como uma documentação viva do que o sistema permite (ou não) fazer.

3. Relatório da Ferramenta

Abaixo está o registro da análise estática gerada pela execução do linter (ESLint) na suíte de testes original:

```
PS D:\Biblioteca\Documentos\GitHub\test-smelly> npx eslint
D:\Biblioteca\Documentos\GitHub\test-smelly\test\userService.smelly.test.js
 44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 77:3  warning  Tests should not be skipped            jest/no-disabled-tests
 77:3  warning  Test has no assertions                 jest/expect-expect

X 6 problems (4 errors, 2 warnings)
```

Como a ferramenta automatizou a detecção:

A execução do ESLint configurado de maneira estrita demonstrou enorme precisão na detecção de falhas estruturais. A ferramenta automatizou a caça aos smells apontando a regra `jest/no-conditional-expect` exatamente nas linhas que continham lógicas condicionais (o bloco `if` do administrador e o bloco `catch` do menor de idade), confirmando de forma imediata o risco de falsos positivos que havíamos identificado na análise manual. Adicionalmente, o linter capturou débitos técnicos instantaneamente, alertando sobre testes marcados para serem pulados (`jest/no-disabled-tests`) e sobre a ineficácia de testes que sequer possuíam asserções (`jest/expect-expect`). Essa automatização atua como um portão de qualidade, detectando vulnerabilidades de arquitetura que facilmente passariam despercebidas em *Code Reviews* manuais.

Após a refatoração:

```
PS D:\Biblioteca\Documentos\GitHub\test-smelly> npm test

> test-smells-lab@1.0.0 test
> jest

PASS test/userService.clean.test.js
PASS test/userService.smelly.test.js

Test Suites: 2 passed, 2 total
Tests:       1 skipped, 11 passed, 12 total
Snapshots:   0 total
Time:        3.103 s
Ran all test suites.
```

4. Conclusão

A prática de garantir a qualidade em projetos de software prova que a busca por testes eficientes vai muito além de uma alta porcentagem de cobertura. Como demonstrado nesta refatoração, uma suíte de testes afetada por *Test Smells* pode ser perigosa: ela se torna frágil perante mudanças cosméticas e irá mascarar problemas graves nas regras de negócio através de asserções silenciosas.

O processo de limpeza e a aplicação de padrões rigorosos, como o AAA e a divisão de escopo focado, transformam o código de teste em uma base de documentação sustentável e de rápida compreensão. Paralelamente, o uso de ferramentas de análise estática como o ESLint consolida a sustentabilidade a longo prazo. O linter atua preventivamente como uma linha de defesa automatizada, padronizando a sintaxe e proibindo a injeção de novos smells na base de código, garantindo que o software permaneça resiliente, manutenível e preparado para ser escalado de maneira segura por toda a equipe.